# Final Project: implementation of a neural calculator

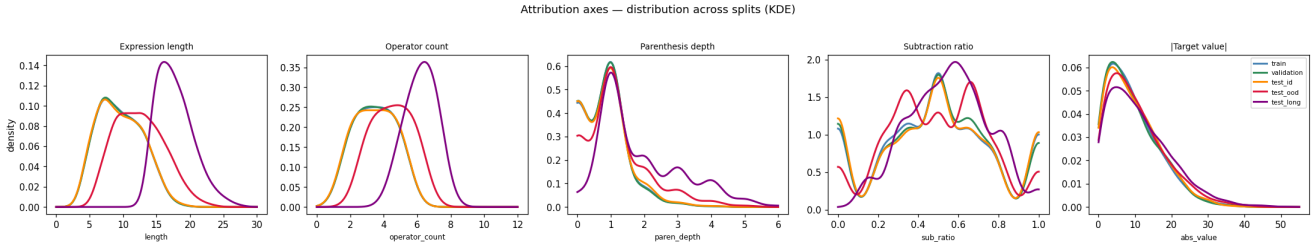
Laura Minonne Filippo Tondini Tsztsun Yang

**Abstract**

This project investigates the generalization capabilities of neural networks on a simple mathematical task: the neural calculator. The primary objective is to train a machine learning model that receives an arithmetic expression as a string and accurately predicts its integer output value. Our main goal is to evaluate generalization beyond the training distribution. Specifically, we focus on the extrapolation behavior of three substantially different architectures—FCNN, CNN, and LSTM. We aim to assess their robustness not only to the sequences observed during training but also to out-of-distribution and long-sequences tests. To improve coverage, we applied custom data augmentation specifically built to improve the models' accuracy on OOD and longer sequences. The best model, a bidirectional LSTM, reaches 98.7% (`test_id`),96.0% (OOD) and 86.7% (long-sequences) exact accuracy.

F. Tondini and L. Minonne jointly implemented the FCNN and CNN, while T. Yang developed the LSTM and the appendixes. Data analysis and augmentation were conducted collaboratively by all members.

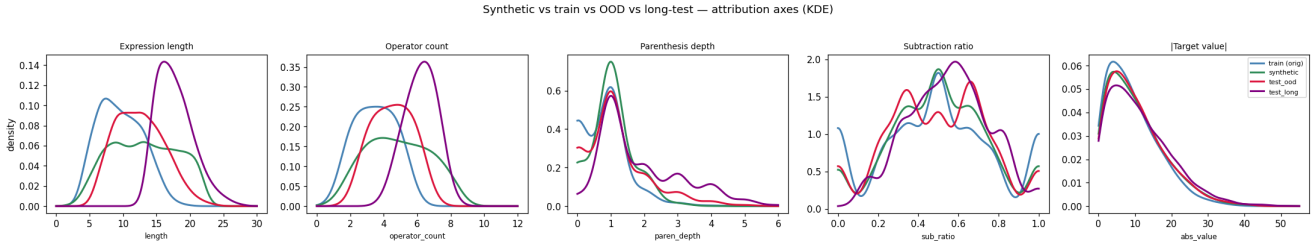
## 1 Data Analysis & Tokenization




Our exploratory data analysis confirms that the Train and In-Distribution (ID) test sets share identical distributions. The Out-Of-Distribution (OOD) and Long test sets are specifically designed to test structural extrapolation, introducing longer sequences, deeper parenthesis nesting, and higher operator counts. While the target value magnitude remains relatively consistent across splits, this structural complexity changes the spatial footprint of the expressions.

To process these strings, we employed a character-level tokenization strategy. We defined a minimal vocabulary of 15 tokens: the ten digits (0-9), the operators, parentheses, and a `<PAD>` token to ensure uniform tensor dimensions across batches. This fixed-dimensional structure is required for architectures like the FCNN and the CNN. As a result, tokenized sequences are padded if short, but truncated if they exceed the predefined maximum length.

## 2 Data Augmentation




To help our models handle the more complex test sets, we generated 36,000 synthetic expressions specifically targeted to fill the 15-21 character length region. To guarantee that our synthetic data provided genuinely new examples, we actively discarded any generated expression that was already present in the original training set. The expressions are built step-by-step starting from a base element, which can be either a single digit or a parenthesized block. The generator then expands the string by randomly adding an operator (+ or -) followed by another digit or new bracketed

| model | data | id | ood | long |
|---|---|---|---|---|
| FCNN | raw | 81.3 | 58.2 | 14.0 |
| FCNN | aug | 61.8 | 41.3 | 12.9 |
| CNN(no-pool) | raw | 99.9 | 94.9 | 65.9 |
| CNN(no-pool) | aug | 98.4 | 92.0 | 71.1 |
| BiLSTM | raw | 91.0 | 81.5 | 61.0 |
| BiLSTM | aug | 98.7 | 96.0 | 86.7 |

**Figure 1:** Raw vs augmented

sub-chain. Every time a new piece is attached to the string, its mathematical value is immediately added to or subtracted from a running total. This step-by-step assembly ensures that we obtain perfectly accurate mathematical labels automatically, eliminating the need for any external parsing tools or post-processing. Even though on raw data CNN beats LSTM, augmented data improves the performance of CNN and LSTM on the extrapolation splits and make LSTM more fit for the task.

**Training setup.** To ensure a fair comparison, all architectures share a consistent training setup. Every model is optimized using the Adam optimizer. Training is conducted with a batch size of 64, employing Mean Squared Error (MSE) as the loss function to guide the regression task.

# 3 The FCNN Model

## 3.1 Architecture and Operating Mechanism

The first implemented model is a Fully Connected Network (FCNN/MLP) designed to map character sequences to continuous numerical values. The architecture begins with an Embedding layer that maps the 15 tokens into vectors. Next, a Flattening operation collapses these vectors into a single 1D representation of length max_len$\times$32. This vector is processed by a network consisting of two hidden layers (256 and 128 units) and a single output, utilizing ReLU activation functions.

## 3.2 Regression vs. Classification

For evaluating mathematical expressions, Regression outperforms Classification, as evidenced by the `test_id` exact match accuracy reaching 75.90% for Regression compared to just 55.20% for Classification. Since the targets range in $[-99, 99]$, Regression treats the space as continuous, using Mean Squared Error to penalize geometric distance and guide numerical interpolation:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$

In contrast, Classification predicts a probability vector across 199 isolated labels via Cross-Entropy:

$$CE = -\sum_{c=1}^{C} y_c \log(\hat{y}_c)$$

This forces the network to treat numbers as geometrically disconnected classes. Consequently, it penalizes minor mistakes (e.g., 44 instead of 45) exactly like massive failures, makes value extrapolation impossible, and drives the FCN architecture into overfitting( 95.78% accuracy on the training set but 54.55% on validation vs the regression model with 82.93% train and 76.25% validation).

## 3.3 Fundamental Architectural Limitations

This design imposes structural limits that preclude the learning of mathematical syntax. The fully connected architecture lacks translation invariance: by flattening the sequence into a fixed-length vector, the network binds its weights to absolute spatial coordinates. This makes it unable to recognize a mathematical sub-expression if it reappears along the expression. Second, simultaneous processing deprives the FCNN of the internal memory needed to track state across recursive operations, such as holding a minus sign for nested parentheses. Finally, the fixed input dimensionality constraint forces the truncation of sequences in the more complex test sets, inevitably degrading predictions.

## 3.4 Performance

The experimental results reflect these limitations: the FCNN achieves good In-Distribution accuracy (82.93%) by memorizing positional combinations on known expressions. However, performance drops on the OOD test (52.60%), where a higher density of subtractions alters the standard positioning of signs, and collapses on the Long test (13.60%) due to truncation and the lack of syntactic generalization.

# 4 The Convolutional Neural Network

The second model we analyzed is the CNN. The first step is the Embedding Layer where the sequences of tokens representing the input expression are mapped to a tensor of dimensions Batch Size (64) $\times$ Sequence Length (21) $\times$ Embedding Dimension (32).

The network then processes the embedded sequences using consecutive 1D Convolutional layers with a fixed kernel size of 5, which we can see as the smallest window spanning an operator with both operands plus a bracket, a common sub-expression. In order to improve the CNN our experiments focused on evaluating the preservation of positional and count information and network depth.

### 4.1 Max-Pooling vs Avg-Pooling vs. No-Pooling

Adding pooling to a CNN makes it length agnostic (trained at `max_len` it evaluates `test_long` at full length 27 without truncation) so it could hypothetically improve the performance, in particular max-pooling extracts only the biggest value across the final sequence while avg-pooling computes the mean activation over the valid sequence length. However, mathematical expression evaluation is highly position-dependent as changing the location of an operator minus or a parenthesis can completely alter the final result. Max pooling pushes the network to learn filters that act as detectors of isolated spikes of activation so it only registers whether a feature is present, completely ignoring how many times it occurred and failing to capture any cumulative summation . On the other hand, even though avg-pooling could theoretically preserve a proportional count of operator activations, many different expressions have a similar average contribution making the representation even worse. With no pooling the final linear layer over the flattened sequence acts as a weighted sum, allowing the network to encode each token's contribution but being truncated after the 21st character. We evaluated three basic CNN architectures, each containing three convolutional layers, where the only structural difference was the final operation: a masked global max pool, a global average pool, or no pooling at all, which was implemented by simply flattening the sequence . The introduction of pooling resulted in a severe general underperformance compared to the no-pooling, especially regarding the OOD and LS tests which only gained 54.25% and 24.20% (max-pool), 37.20% and 18.00% (avg-pool) compared to the No-Pool respective ones, 85.15% and 58.80%.

### 4.2 Network Depth and Receptive Field Expansion

We then examined the optimal depth of the convolutional encoder by adding more layers to the No-pool model. The theoretical Receptive Field ($RF$) of a 1D convolutional stack with kernel size $k$ and $L$ number of layers can be calculated as:

$$RF_L = RF_{L-1} + (k-1)$$

However, this formula defines the absolute span of the convolutional window. At the sequence boundaries the effective receptive field over valid input tokens is smaller, as the window partially overlaps with the two zero-padded regions required for the kernel size.

The transition from 3 layers to 4 layers, which expands the receptive field from 13 tokens up to 17 tokens, resulted in a general increase of accuracy for every test, with the OOD and LS splits reaching 92.00% and 71.13%. The receptive larger window allows deeper convolutional features to represent fully nested sub-expressions before flat projection. The receptive formula suggested that increasing by one more layer the model should improve as the window covers 21 tokens but we can see a decrease in performance (especially regarding the long sequences test accuracy dropping at 63.00%) showing a slight occurance of what was first named in 2015 as the Degradation Problem (He et al., 2015).

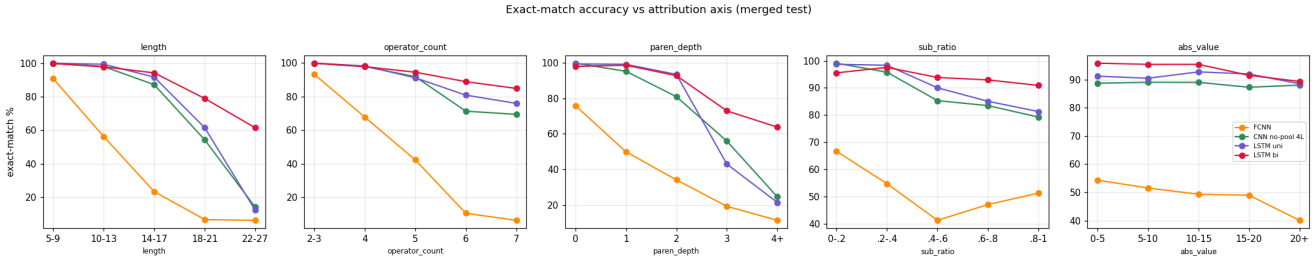
## 5 LSTM (UniLSTM and BiLSTM)

After the no-pool CNN's fixed-window limitation, we turn to a recurrent model (Zhang et al., 2023, cap. 10). Reading tokens one at a time, LSTM is length agnostic like pooling, yet preserves the full ordered sequence like no-pool, with an unbounded receptive field. It maintains a cell state (long-range memory across the expression) and a hidden state (short-term memory read out through the output gate); we use 2 layers of 128 hidden units and pass the last layer's final hidden state to a small FCNN that outputs the scalar prediction.

**Why bidirectional.** Even unidirectionally the LSTM already beats the best CNN, so the remaining choice to improve the model is the reading direction. As this is an encoding task — the whole expression is given and we only output its value — there is no reason to force a causal left-to-right pass, as one would for generation; a bidirectional LSTM reads each layer both forward and backward and concatenates the two final hidden states, so every position's representation is built from the entire sequence rather than only its prefix.

**Why it helps.**The bidirectional model beats the unidirectional one on every split, with the gain concentrated on deep and long expressions (OOD 96.0% vs 93.3%, long 86.7% vs 78.5%). This follows from the data's structure: every parenthesis is a subtraction, so all nesting is right-nested subtraction a-(b-(c-(d-e))), whose sign-fixing innermost group sits at the end of the string. A forward-only pass must therefore carry a growing stack of pending sign-flips and resolve them only at the end, which degrades as nesting deepens; the bidirectional model's backward pass instead reads that innermost group first and resolves outward. The account makes a concrete prediction: a purely backward (right-to-left) LSTM should, at equal capacity, outperform the forward one on exactly these deep expressions — indicating the advantage is reading direction, not capacity.

# 6 Conclusions and Final Comparisons




## 6.1 On Long Sequences

To preserve positional context without destroying arithmetic structure both FCNN and CNN(No-Pool) required a `Flatten` layer. This structurally locks the network to a maximum sequence length (21) which leads to arbitrary truncation in longer sequences. On the other hand LSTM recursively updates a single memory vector and outputs only the final summary state ($h_n$), the network's output dimension is entirely independent of the input length. The LSTM naturally generalizes to the `test-long` split without any truncation. However, while in UniLSTM early token information fades across too many recurrent steps, in BiLSTM information distant in one pass becomes closer in the reverse. From the graph it is clear that apart from BiLSMT every other model's accuracy completely collapses in the range 22-27 characters.

## 6.2 Parenthesis Nesting and Subtraction Ratio

As FCNN flattens the entire input sequence before processing, every learned pattern is tightly bound to a specific absolute position so no pattern such as parenthesis nesting nor operator is recognised. This limitation could be also seen in the appendix as we saw that on the augmented data FCNN underperforms, unlike CNN and LSTM. CNN uses convolutions to read local windows, understanding local patterns but even at the optimal 4-layer depth, the network is restricted by a fixed receptive field of 17 tokens. If a parenthesis block exceeds this length, the CNN doesn't see the relationship between the opening and closing brackets or non unary minus before a parenthesis. In addition we can see as expression complexity (depth over 3) increases, the associated string length exceeds max length causing truncation. Although LSTMs can track deep parenthesis nesting and keep a running total of sign parity, the graph shows uniLSTM underperforms the optimal 4-layer CNN on parenthesis nesting because of the dataset's heavy presence of right-nested subtractions and as a consequence many stacked parentheses at the end of the expression. An only left-to-right forward reading forces the network to stack every enclosing parenthesis into memory.

## 6.3 Magnitude of the result

The graph shows that LSTM and CNN(no-pool) maintain an accuracy of approximately 90% on targets larger than 20, while FCNN fails completely. Since the vocabulary is limited to single digits, reaching a high-magnitude target requires adding multiple operands together, often resulting in longer sequences. Both the LSTM and the CNN(no-pool) are capable of accumulating these values: the LSTM tracks the running total within its cell state, while the CNN(no-pool) utilizes its final linear layer to compute a precise weighted sum over the translation-invariant features extracted by the convolutions. In contrast, FCNN collapses due to a combination of length truncation and a lack of translation invariance, which prevents it from generalizing shifted additive patterns.

| **Model** | **Validation** Acc / MAE / RMSE | **Test ID** Acc / MAE / RMSE | **Test OOD** Acc / MAE / RMSE | **Test Long** Acc / MAE / RMSE |
|---|---|---|---|---|
| FCNN | 76.2 / 0.373 / 0.583 | 75.9 / 0.385 / 0.601 | 52.6 / 1.170 / 2.324 | 13.6 / 3.799 / 5.326 |
| CNN max-pool 3L | 74.2 / 0.408 / 0.653 | 73.9 / 0.420 / 0.650 | 54.2 / 0.873 / 1.680 | 24.2 / 2.187 / 3.667 |
| CNN np 3L | 95.8 / 0.168 / 0.227 | 95.0 / 0.171 / 0.235 | 85.2 / 0.326 / 0.639 | 58.8 / 0.738 / 1.282 |
| CNN np 4L | 98.9 / 0.129 / 0.172 | 98.4 / 0.138 / 0.189 | 92.0 / 0.263 / 0.635 | 71.1 / 0.645 / 1.238 |
| LSTM uni. | 98.9 / 0.111 / 0.178 | 98.4 / 0.112 / 0.167 | 93.3 / 0.206 / 0.455 | 78.5 / 0.513 / 1.128 |
| BiLSTM | 99.5 / 0.170 / 0.206 | 98.7 / 0.179 / 0.218 | 96.0 / 0.203 / 0.261 | 86.7 / 0.290 / 0.521 |

**Table 1:** Performance comparison: Exact-match accuracy (%), Mean Absolute Error (MAE), and Root Mean Squared Error (RMSE).

After analyzing the fixed spatial constraints of the FCNN and the local pattern recognition of the CNN, our findings show that a more flexible form of sequential memory is required. BiLSTM remains the most stable across all splits, overcoming the degradation caused by increasing structural complexity, length, and parenthesis depth which are inevitably correlated as longer ⇒ deeper ⇒ more operators.